

```

import argparse
import cv2
import numpy as np
import math
import matplotlib.pyplot as plt

"""
Hough transform function to detect lines from the image.

Input:
    img - Full path of the input image.
    num_rho - No. of Rhos in Rho-Theta Hough Space. Default 180.
    num_theta - No. of Thetas in Rho-Theta Hough Space. Default 180.
    bin_threshold - bin / vote values above or below the bin_threshold
are shortlisted as lines. Default 150.

Return:
    line_img - Image with the Lines drawn
    lines - List of lines in format (rho,theta,x1,y1,x2,y2)
"""

def find_hough_lines(image, edge_image, num_rhos, num_thetas,
bin_threshold):

    img_height, img_width = edge_image.shape[:2]
    img_height_half = img_height / 2
    img_width_half = img_width / 2

    # Rho and Theta ranges
    diag_len = np.sqrt(np.square(img_height) + np.square(img_width))
    dtheta = 180 / num_thetas
    drho = (2 * diag_len) / num_rhos

    ## Thetas is bins created from 0 to 180 degree with increment of the
provided dtheta
    thetas = np.arange(0, 180, step=dtheta)

    ## Rho ranges from -diag_len to diag_len where diag_len is the
diagonal length of the input image
    rhos = np.arange(-diag_len, diag_len, step=drho)

    #
    cos_thetas = np.cos(np.deg2rad(thetas))
    sin_thetas = np.sin(np.deg2rad(thetas))

    # Hough accumulator array of theta vs rho, (rho,theta)
    accumulator = np.zeros((len(rhos), len(thetas)))

    #

```

```

figure = plt.figure()
hough_plot = figure.add_subplot()
hough_plot.set_facecolor((0, 0, 0))
hough_plot.title.set_text("Hough Space")

# Iterate through pixels and if non-zero pixel process it for hough
space
for y in range(img_height):
    for x in range(img_width):
        if edge_image[y][x] != 0:
            edge_pt = [y - img_height_half, x - img_width_half]
            hough_rhos, hough_thetas = [], []

            # Iterate through theta ranges to calculate the rho values
            for theta_idx in range(len(thetas)):
                # Calculate rho value
                rho = (edge_pt[1] * cos_thetas[theta_idx]) + (edge_pt[0] *
sin_thetas[theta_idx])
                theta = thetas[theta_idx]

                # Get index of nearest rho value
                rho_idx = np.argmin(np.abs(rhos - rho))

                #increment the vote for (rho_idx,theta_idx) pair
                accumulator[rho_idx][theta_idx] += 1

            #
            hough_rhos.append(rho)
            hough_thetas.append(theta)

        #
        hough_plot.plot(hough_thetas, hough_rhos, color="white",
alpha=0.05)

#
output_img = image.copy()
# Output list of detected lines. A single line would be a tuple of
(rho,theta,x1,y1,x2,y2)
out_lines = []

for y in range(accumulator.shape[0]):
    for x in range(accumulator.shape[1]):
        #
        if accumulator[y][x] > bin_threshold:
            rho = rhos[y]
            theta = thetas[x]

            # a and b are intercepts in x and y direction

```

```

a = np.cos(np.deg2rad(theta))
b = np.sin(np.deg2rad(theta))

x0 = (a * rho) + img_width_half
y0 = (b * rho) + img_height_half

# Get the extreme points to draw the line
x1 = int(x0 + 1000 * (-b))
y1 = int(y0 + 1000 * (a))
x2 = int(x0 - 1000 * (-b))
y2 = int(y0 - 1000 * (a))

# Plot the Maxima point on the Hough Space Plot
hough_plot.plot([theta], [rho], marker='o', color="yellow")

#
1) output_img = cv2.line(output_img, (x1,y1), (x2,y2), (0,255,0),

# Add the data for the line to output list
out_lines.append((rho,theta,x1,y1,x2,y2))

#
hough_plot.invert_yaxis()
hough_plot.invert_xaxis()
plt.show()

return output_img, out_lines

def peak_votes(accumulator, thetas, rhos):
    """ Finds the max number of votes in the hough accumulator """
    idx = np.argmax(accumulator)
    rho = rhos[int(idx / accumulator.shape[1])]
    theta = thetas[idx % accumulator.shape[1]]

    return idx, theta, rho

def theta2gradient(theta):
    """ Finds slope m from theta """
    return np.cos(theta) / np.sin(theta)

def rho2intercept(theta, rho):
    """ Finds intercept b from rho """
    return rho / np.sin(theta)

image_paths = ['imgs/ex1.png', 'imgs/ex2.png', 'imgs/ex3.png']
images = [cv2.imread(path) for path in image_paths]

```

```

for i, img in enumerate(images):
    plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(f'Original Image {i+1}')
    plt.show()

num_rho = 180
num_theta = 180
bin_threshold = 150

for i, img in enumerate(images):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, edge_image = cv2.threshold(gray, 120, 255,
cv2.THRESH_BINARY_INV)

    line_img, lines = find_hough_lines(img, edge_image, num_rho,
num_theta, bin_threshold)

    plt.imshow(cv2.cvtColor(line_img, cv2.COLOR_BGR2RGB))
    plt.title(f'Custom Hough Transform - Image {i+1}')
    plt.show()

def hough_lines_cv(image):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    edges = cv2.Canny(gray, 50, 150, apertureSize=3)

    lines = cv2.HoughLinesP(edges, 1, np.pi/180, threshold=100,
minLineLength=5, maxLineGap=10)

    line_img = image.copy()
    if lines is not None:
        for line in lines:
            x1, y1, x2, y2 = line[0]
            cv2.line(line_img, (x1, y1), (x2, y2), (0, 255, 0), 2)

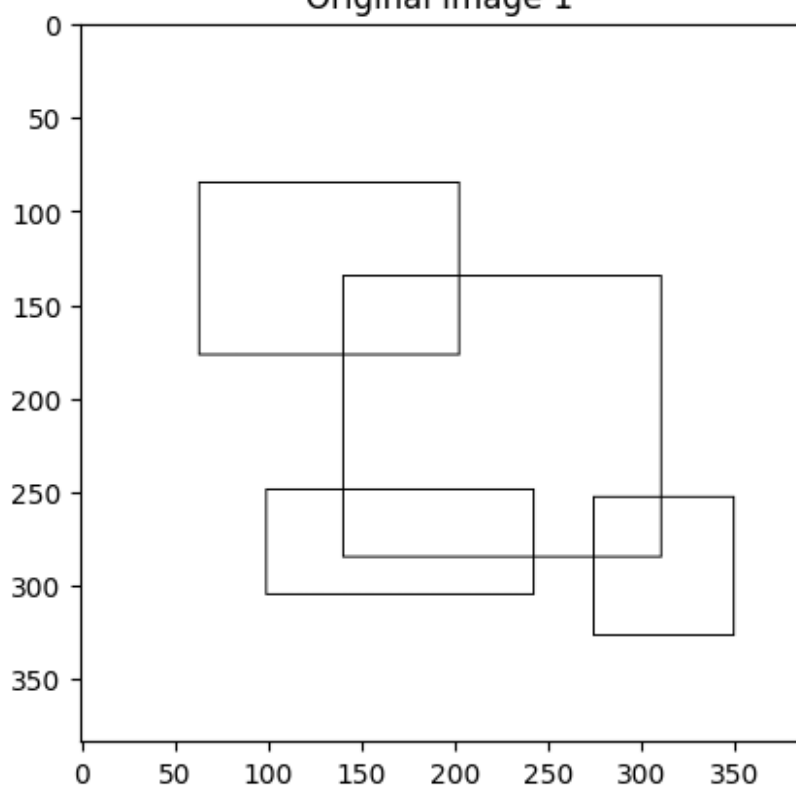
    return line_img

for i, img in enumerate(images):
    line_img_cv = hough_lines_cv(img)

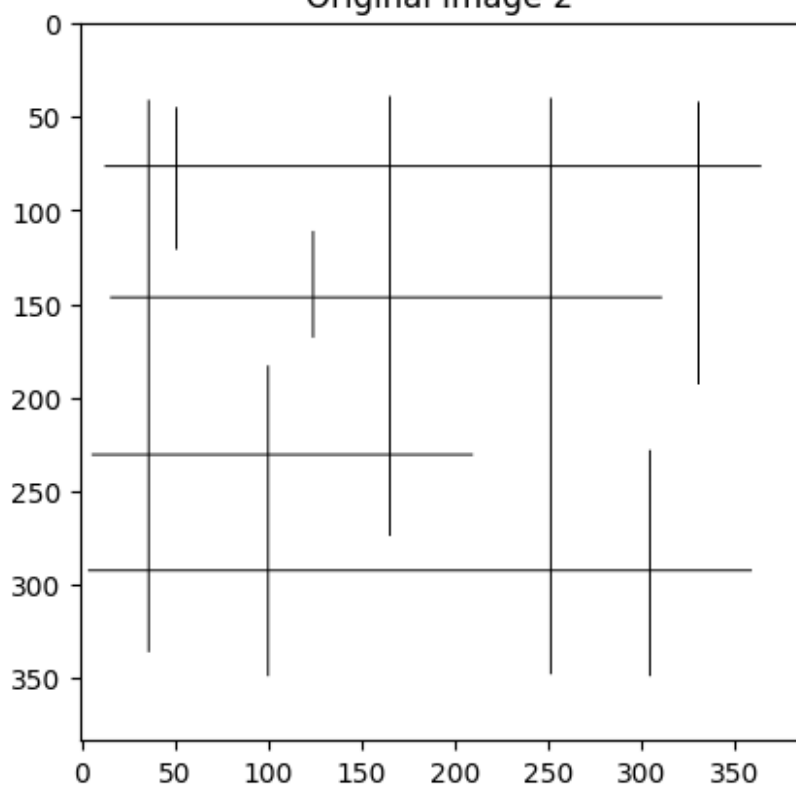
    plt.imshow(cv2.cvtColor(line_img_cv, cv2.COLOR_BGR2RGB))
    plt.title(f'OpenCV Hough Transform - Image {i+1}')
    plt.show()

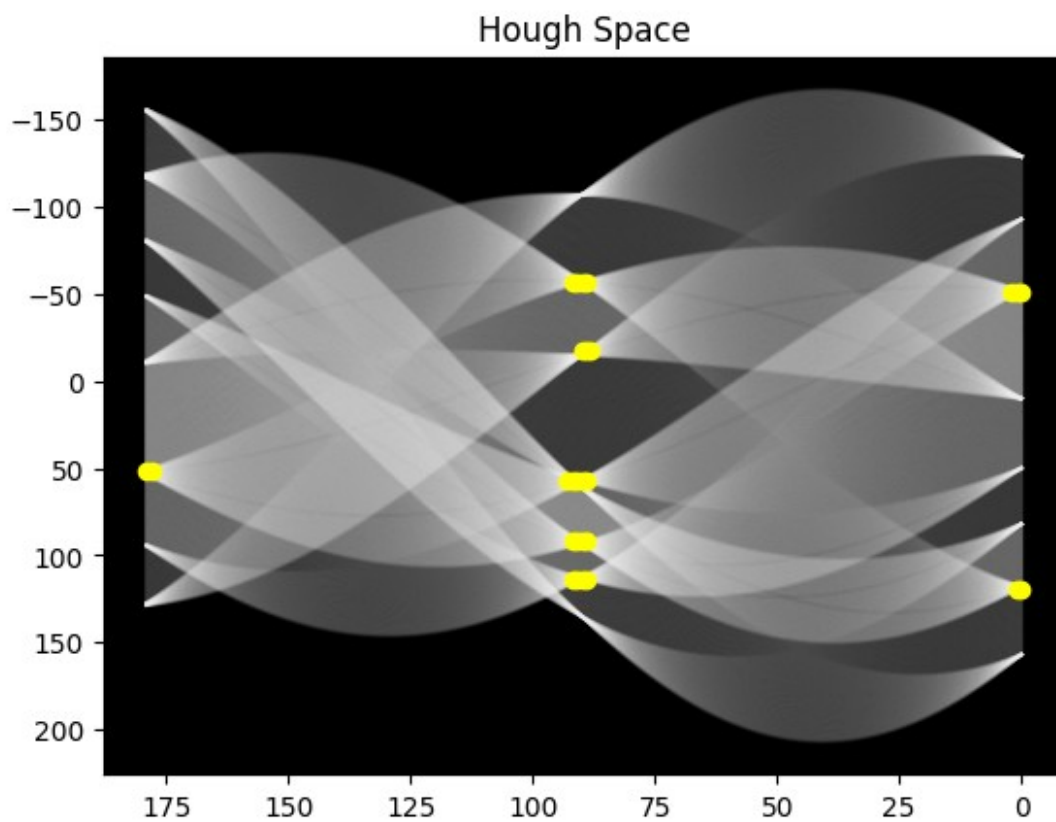
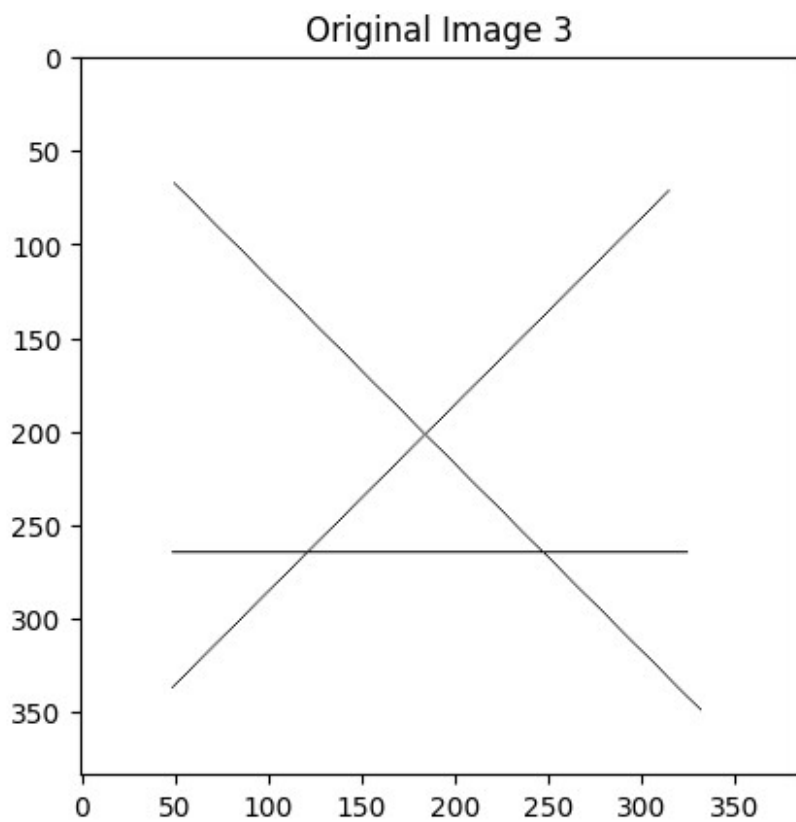
```

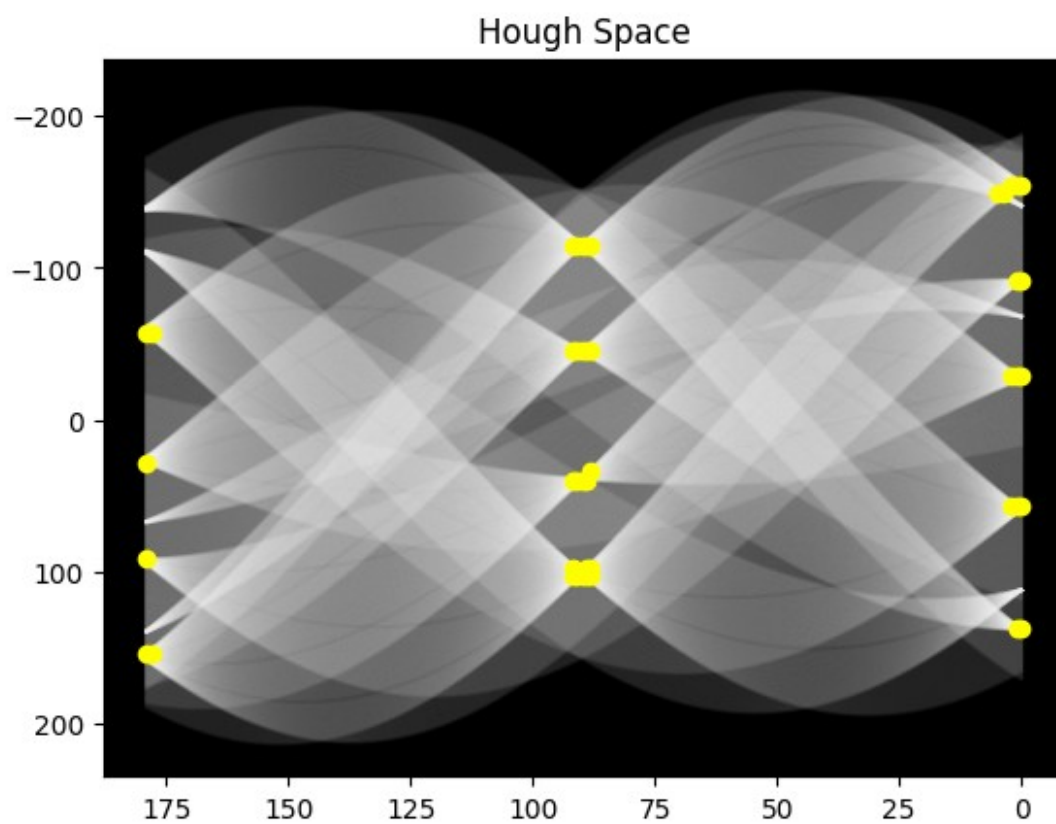
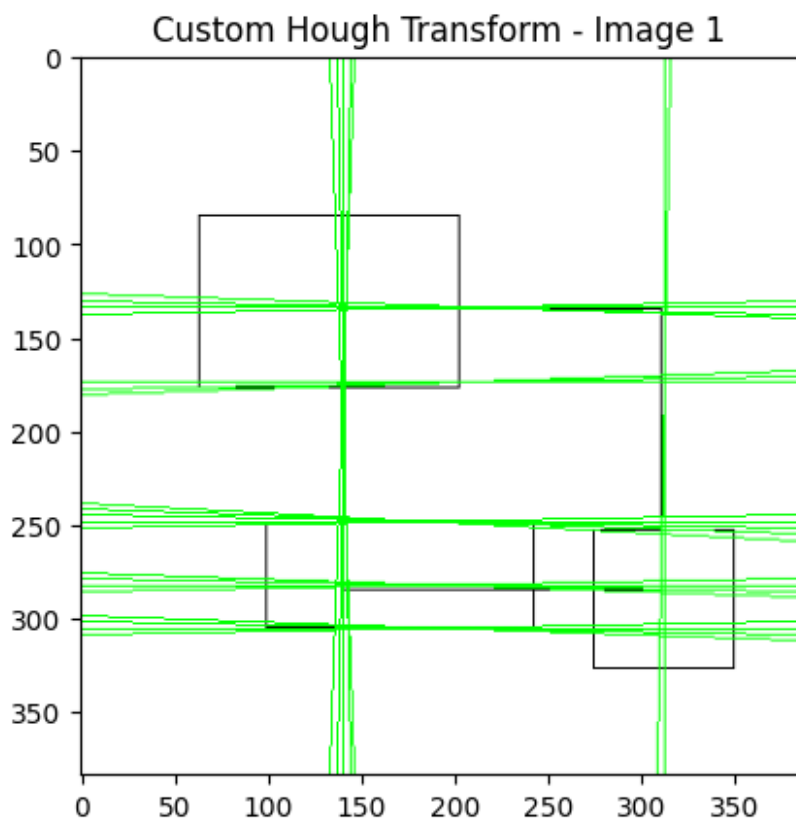
Original Image 1

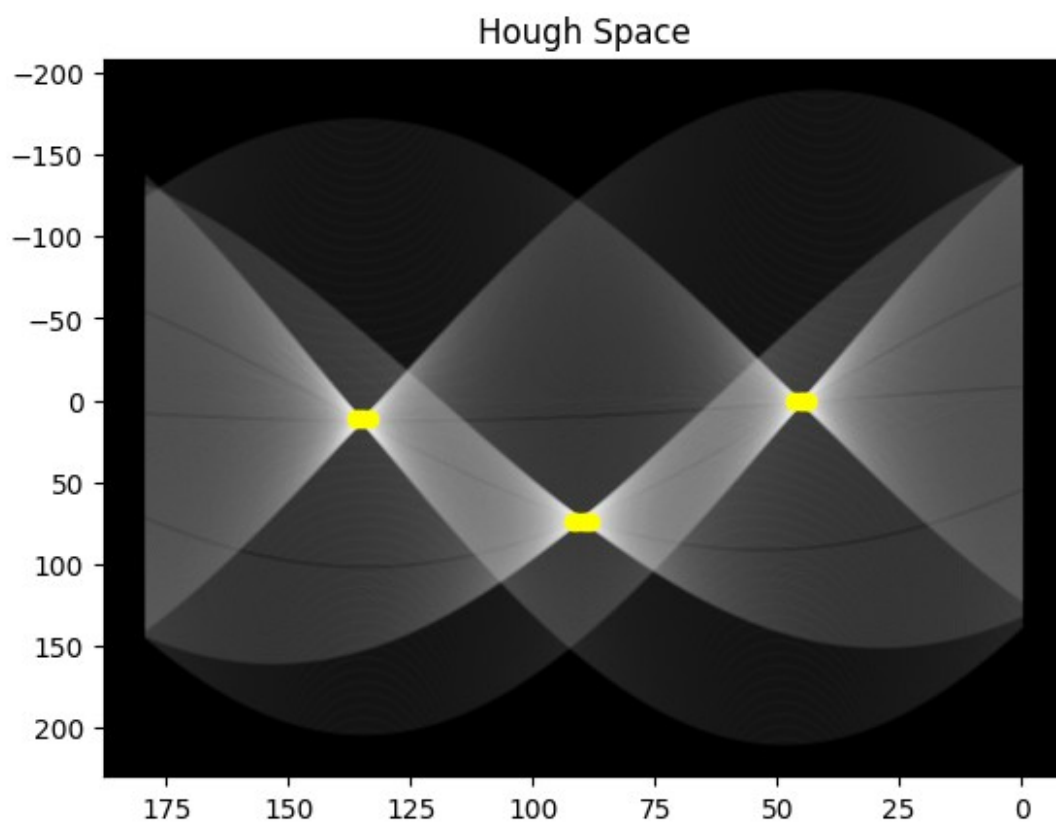
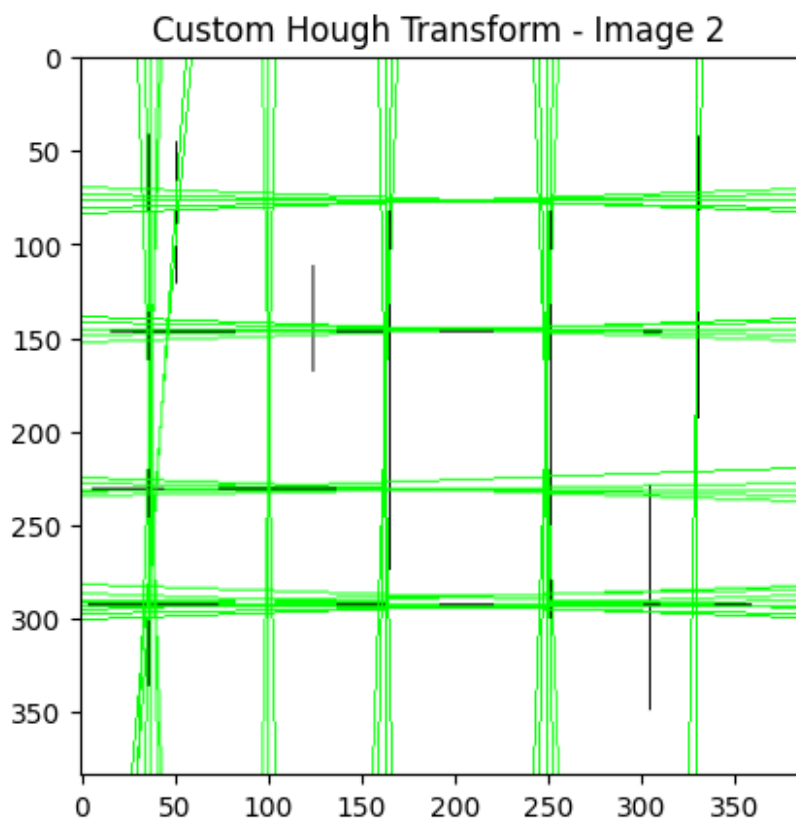


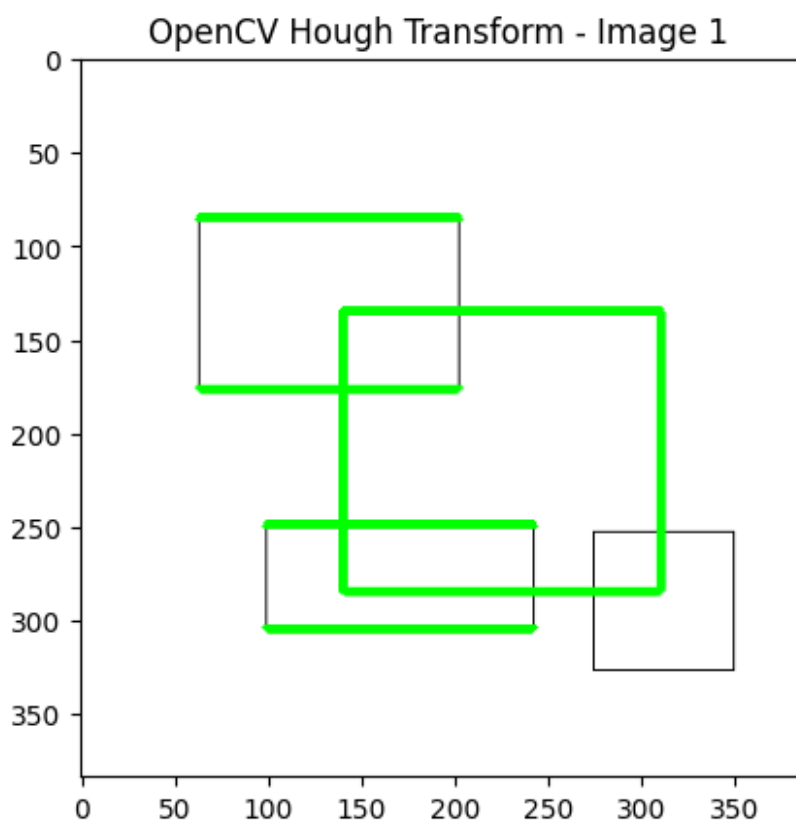
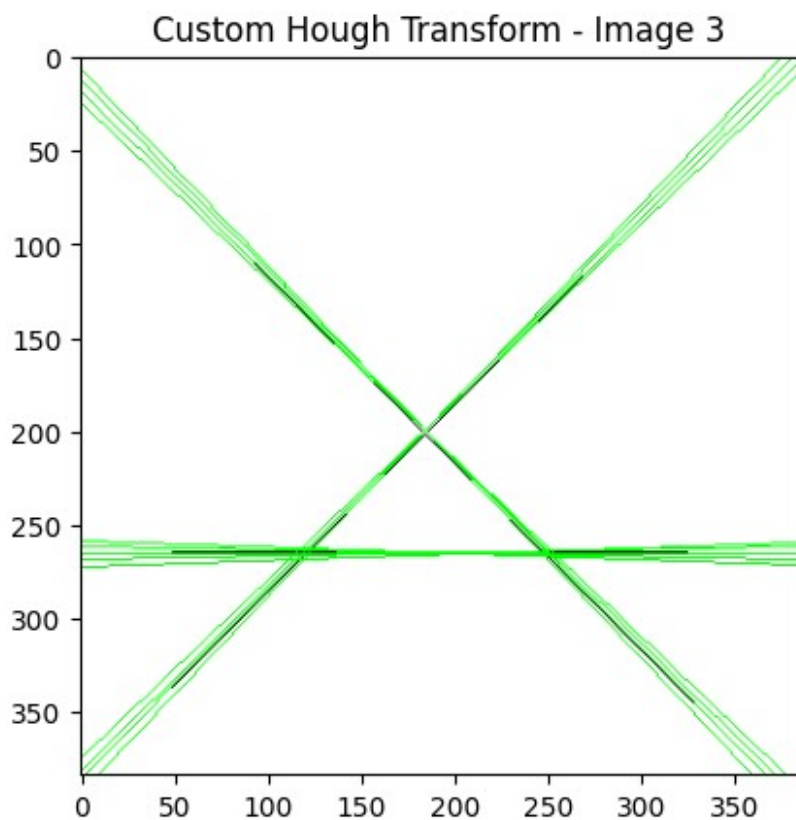
Original Image 2

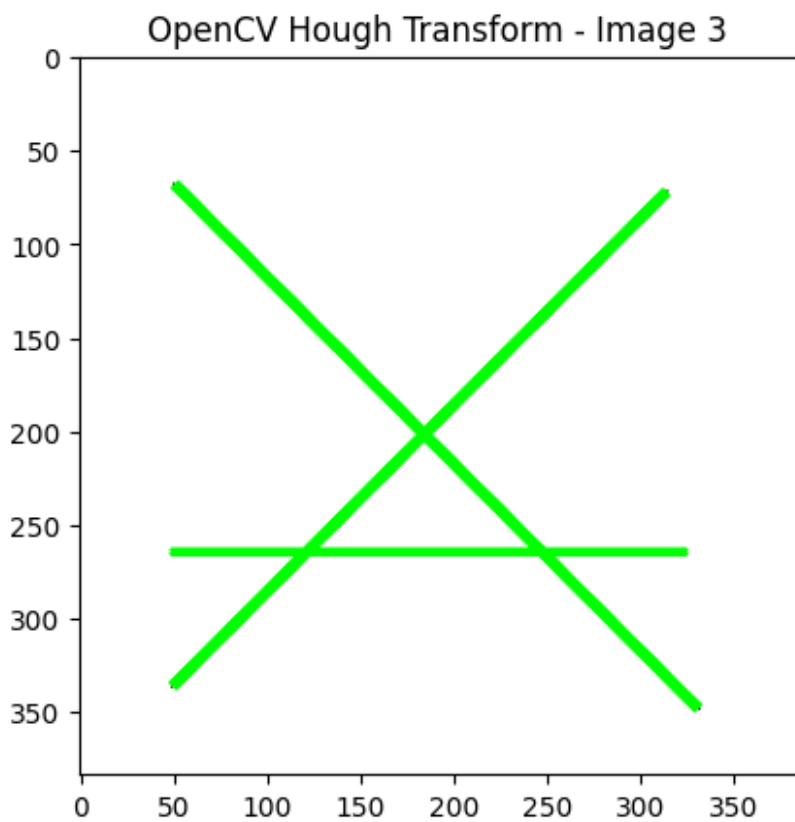
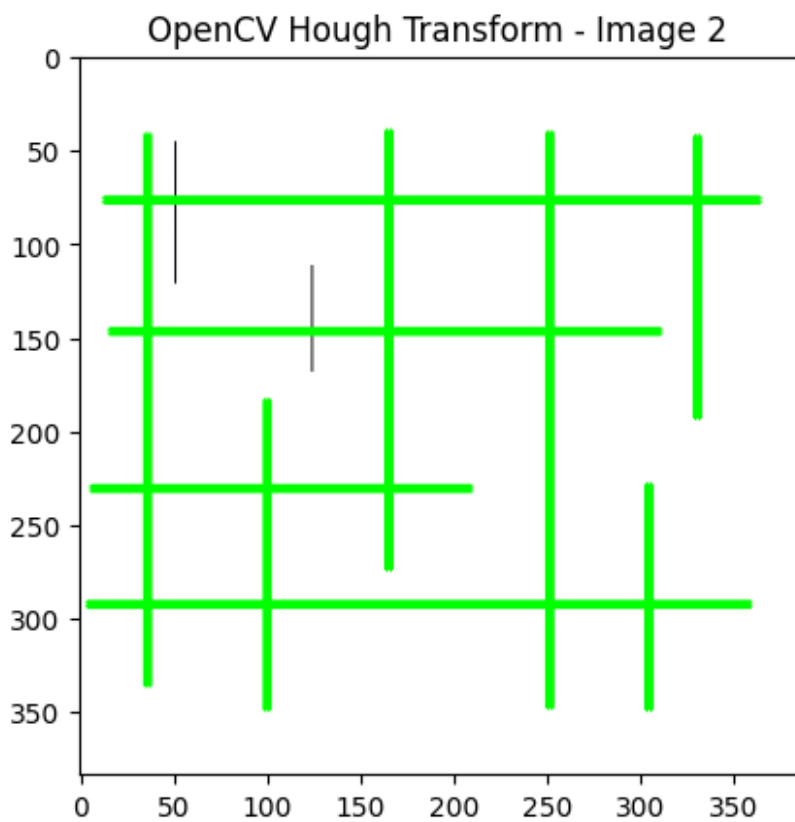












Na powyższych wykresach przedstawiono porównanie działania transformaty Hough podanej na zajęciach oraz z biblioteki OpenCV. Rozwiązanie nie wykorzystujące biblioteki widocznie odstaje jakością od tego z OpenCV. Niektóre linie są mniej dokładnie wykrywane, natomiast gotowa algorytm openCV niemalże idealnie pokrywa wykryte linie.